

5COSC019W – Object Oriented Programming Week 10

Dr. Barbara Villarini

b.villarini@westminster.ac.uk

FILE HANDLING



Introduction

- Storage of data in variables and arrays is temporary
 - when the program is done running or when computer is turned off, the data is gone!
- Files used for long-term retention of large amounts of data, even after the programs that created the data terminate
- Persistent data – exists beyond the duration of program execution
- Files stored on secondary storage devices (hard disks, cloud, etc.)
- Requirements information stored in secondary memory
 - can be retrieved in the future
 - kept separate from other documents, programs, etc.

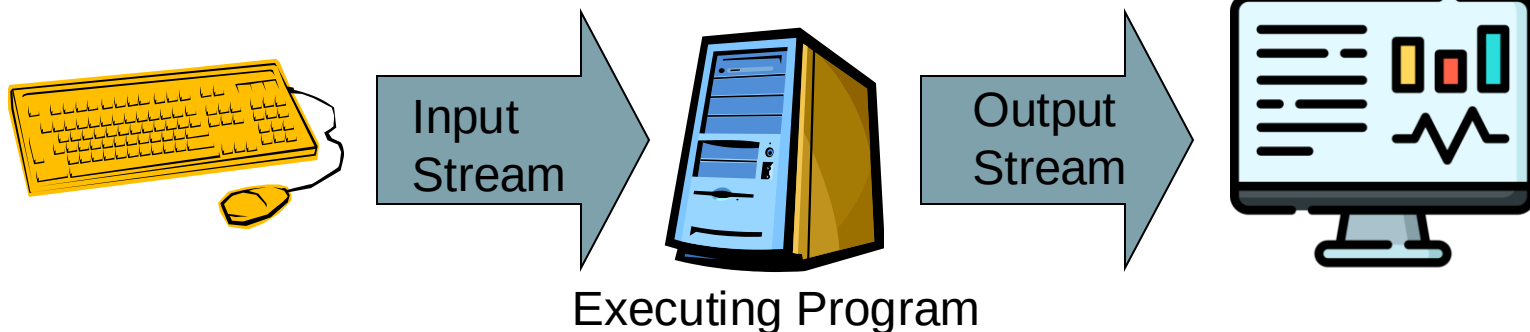
Intro Example: Stock Market Analysis

- Problem: large amounts of stock prices/indices are recorded and processed
 - taken every day
 - stored in a text file **dataStockMarket.dat**
- Analyzing this data means find minimum, maximum, and average of the data
- Data must be read from file, statistics calculated, results written to a text output file



Java's I/O System

- All input and output in Java is accomplished by classes called **streams**
- The purpose of the stream abstraction is to keep program code independent from physical devices.





Type of Stream

- There are 2 kinds of streams
 - byte streams: stores data in binary format
 - character streams: stores data as a sequence of characters
- Java opens file by creating an object and associating a stream with it
- Java libraries define a number of stream classes (`java.io` classes):
 - **Reader** and **Writer** for dealing with character formatted data (e.g., unicode characters).
 - **InputStream** and **OutputStream** for dealing with unformatted data (bytes)

Note that the classes **InputStream**, **OutputStream**, **Reader**, and **Writer** are *abstract* classes.

Predefined Stream

- Three stream objects are automatically created for every application: `System.in`, `System.out`, and `System.err`.
- `System.in`
 - **`InputStream`** object, usually for the keyboard
- `System.out`
 - a buffered **`PrintStream`** object, usually the screen or an active window
- `System.err`
 - an unbuffered **`PrintStream`** object usually associated with the screen or console window

File Class

- Useful for retrieving information about files and directories from disk
- Object of class File doesn't open the actual files/directories.
- It provides methods to operate on the files/directories named.



Create a File object

Creating a File object specifies name/path only:

```
File myDirectory = new File("/Desktop/directory");
```

```
File myFile = new File("/Desktop/directorydataset.txt");
```

```
File myFile = new File("/Desktop/directory", "dataset.txt");
```

```
File myFile = new File(myDirectory, "dataset.txt");
```

Methods to manipulate Files and Directory

- `exists()`
- `isDirectory()`
- `isFile()`
- `canRead()`
- `canWrite()`
- `isHidden()`
- `getName()`
- `getPath()`
- `getAbsolutePath()`
- `getParent()`
- `list()`
- `length()`
- `renameTo( newPath )`
- `delete()`
- `mkdir()`
- `createNewFile()`

FileReader

- Subclass of Reader
- FileReader opens file for reading and throws exception if open fails.

Constructors:

```
FileReader reader = new FileReader("filename");  
FileReader reader = new FileReader(fileObject);
```

- Provides basic set of methods for reading:
 - read character (mapped to character set).
 - read array of characters.
- Also provides method to close stream.

FileWriter

- Subclass of Writer
- FileWriter opens file for writing and throws exception if open fails
- Constructors:

```
FileWriter writer = new FileWriter("filename");
```

```
FileWriter writer = new FileWriter(fileObject);
```

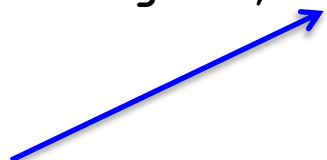
- Provides basic set of write methods:
 - write character.
 - write array of characters.
 - write String.
- Also provides method to close stream.
- If you don't close the file some data may not be written to file.

Remember

- All FileReader/Writer methods throw exceptions. You should use try/catch blocks or write methods with a throws declaration.
- If you open an existing file for writing, you *delete* existing contents, unless *append mode* is selected:

```
FileWriter writer = new FileWriter(fileObject, true);
```

boolean append_to_file





Example – Copying a text file

```
import java.io.*;

public class CopyTextFile {

    public static void main(String[] args) throws IOException {

        File inputFile = new File("text.txt");
        File outputFile = new File("text_copy.txt");

        FileReader in = new FileReader(inputFile);
        FileWriter out = new FileWriter(outputFile);

        int c;
        while ((c = in.read()) != -1) {
            out.write(c);
        }
        in.close();
        out.close();
    }
}
```

Copy one character at a time. read() returns -1 when no more data.

Reading

Writing

Reading/writing files using Byte Streams

- Java methods to read and write bytes from and to a file:
 - **FileInputStream** subclass of `InputStream`
 - **FileOutputStream** subclass of `OutputStream`
- Create an object of one of these classes to open a file
- Specify the name of the file as argument to the constructor.
- Once the file is open you can read/write it



Example – copy a file as bytes

```
import java.io.*;

public class CopyBytesFile {
    public static void main(String[] args) throws IOException {

        File inputFile = new File("Text.txt");
        File outputFile = new File("Text_copy.txt");

        FileInputStream in = new FileInputStream(inputFile);
        FileOutputStream out = new FileOutputStream(outputFile);

        int c;
        while ((c = in.read()) != -1) {
            out.write(c);
        }
        in.close();
        out.close();
    }
}
```

Type int
used to
store
byte.

Copy one byte at a
time.

BufferedReader and BufferedWriter Class

- An object of these classes reads/writes an entire line of text into a buffer
- The constructor take as parameter an object of type Reader
- It provides a **readLine** method to read complete line into a String

```
BufferedReader in = new BufferedReader(new FileReader("data.txt"));  
String s = in.readLine();
```

- String can then be converted to other types (int, double, etc.)



Example – Copying a file

```
import java.io.*;

public class ReadString{
    public static void main(String[] args) throws IOException {

        FileReader in = new FileReader("File1.txt");
        BufferedReader br = new BufferedReader(in);

        FileWriter out = new FileWriter("File2.txt", true);
        BufferedWriter bw = new BufferedWriter(out);

        String line = br.readLine();
        while(line != null){
            bw.write(line);
            bw.newLine();
            line = br.readLine();
        }
        in.close();
        br.close();
        bw.close();
        out.close();

    }
}
```

Use readLine() from
BufferedReader to read
a line of text

Other Reader/Writers

- **StringReader, StringWriter**: read/write to/from strings rather than files or writers.
- **InputStreamReader**: Convert from stream to reader.
- **OutputStreamWriter**: Convert from writer to stream.

Plus a family of `InputStream` and `OutputStream` classes.



Summary

- We saw file handling in Java
- Remember that it is important to save data in files and permanent storage.
- Reading and writing data from and to a file is a must for a good developer!

MULTITHREADING



Overview

- Multithreading
- Threads in Java
- Thread class and Runnable interface
- Concurrency
- Synchronization



Threaded Applications

- Modern Systems
 - Multiple applications run concurrently!
 - This means that... there are multiple processes on your computer





Threaded Applications

- Modern Systems
 - Applications perform many tasks at once!
 - This means that... there are multiple threads within a single process.





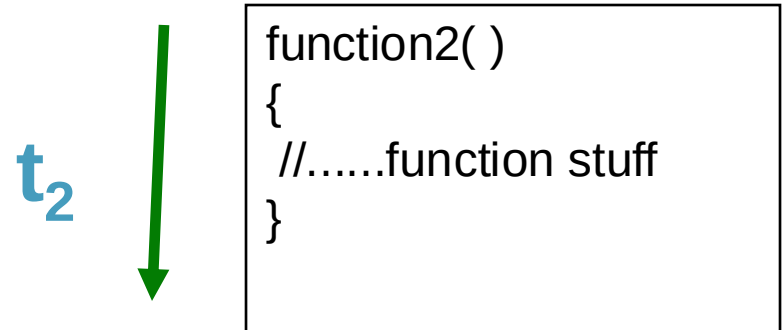
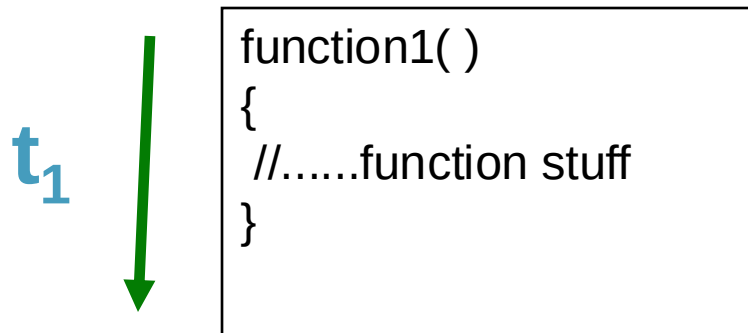
Thread

- Thread: single sequential flow of control within a program
- Single-threaded program can handle one task at any time.
- Multitasking allows single processor to run several concurrent threads.
- Most modern operating systems support multitasking.

Serial vs. Parallel



Serial vs. Parallel in Computing



Serial Machine

```
function1 ( ):  
function2 ( ):  
| Single CPU  
Time : add ( $t_1$ ,  $t_2$ )
```

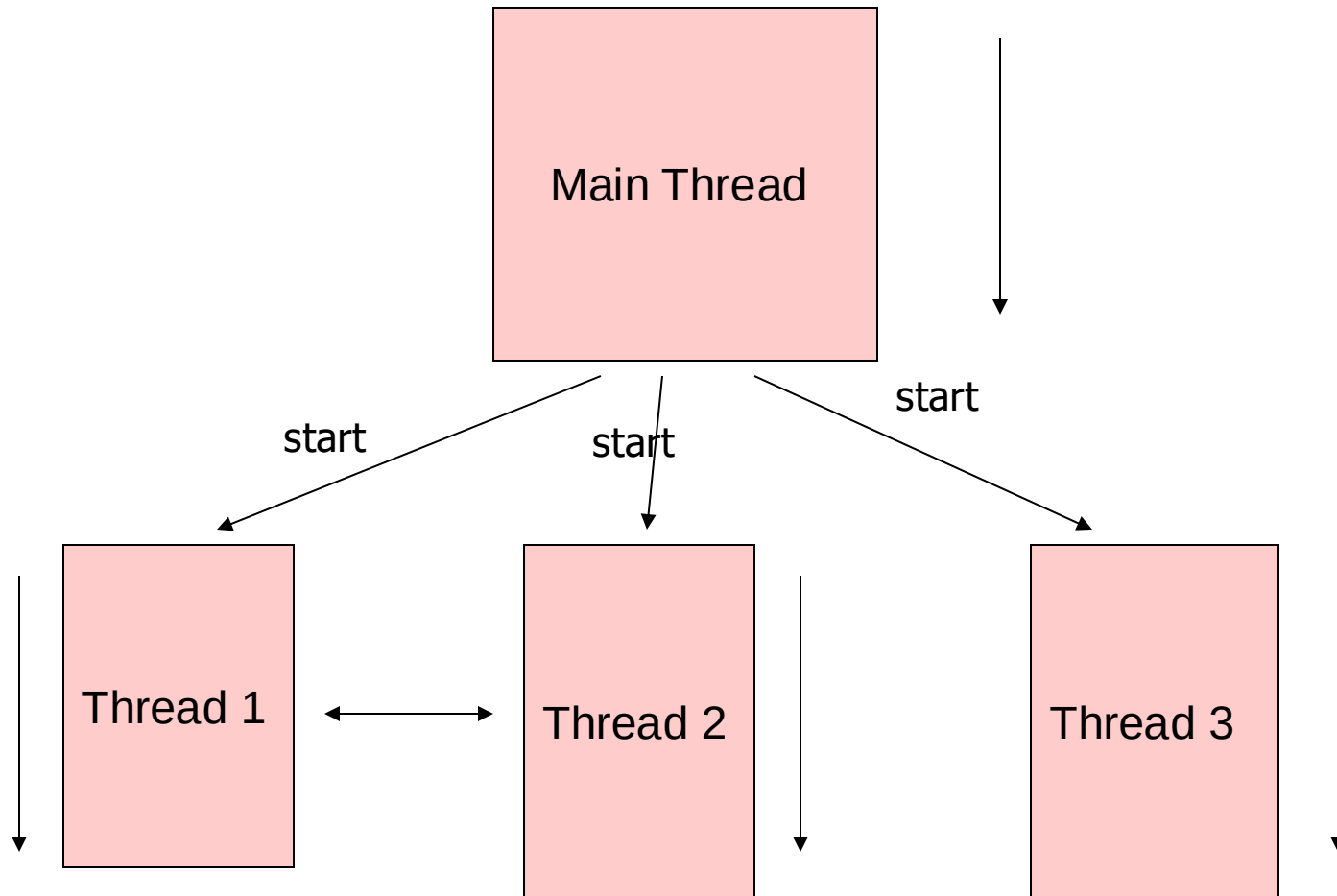
Parallel Machine : MPP

```
function1( ) || function2 ( )  
| massively parallel system  
  containing thousands of CPUs  
Time : max ( $t_1$ ,  $t_2$ )
```

A multithread program



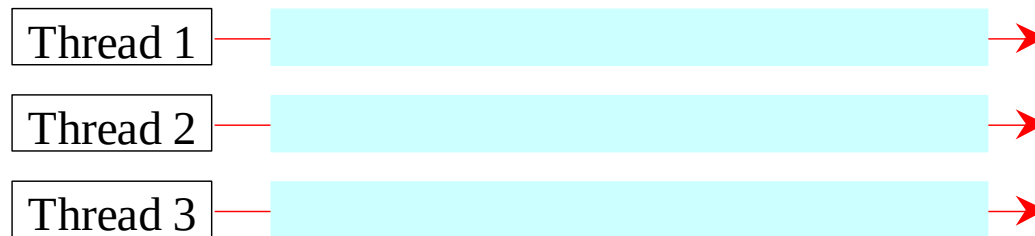
threads are light-weight processes within a process



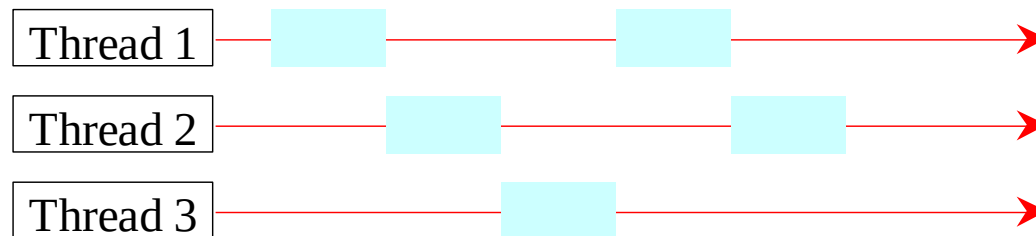
Threads may switch or exchange data/results

Threads Concept

Multiple threads on multiple CPUs



Multiple threads sharing a single CPU





Summing Up

- A Thread is a piece of code that runs in concurrent with other threads.
- Each thread is a statically ordered sequence of instructions.
- Threads are used to express concurrency on both single and multiprocessors machines.
- Programming a task having multiple threads of control – Multithreading or Multithreaded Programming.

Threads in Java

- Java has built in support for Multithreading
- Synchronization
- Thread Scheduling
- Inter-Thread Communication:

currentThread

start

setPriority

yield

run

getPriority

sleep

stop

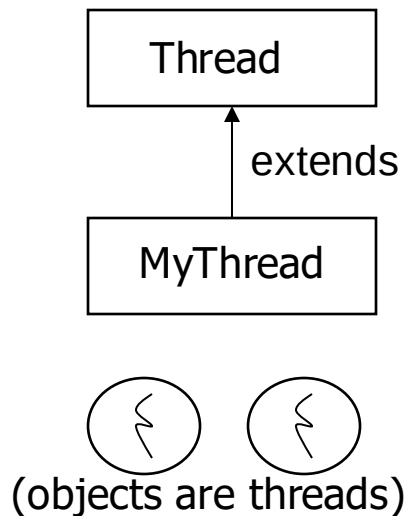
suspend

resume

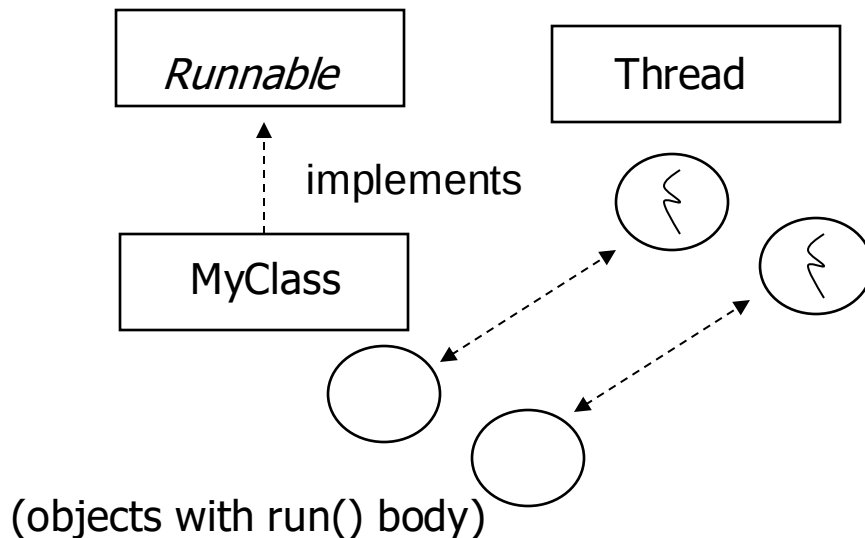
- Java Garbage Collector is a low-priority thread.

2 Threading Mechanisms

1. Create a class that extends the `java.lang.Thread` class
2. Create a class that implements the `java.lang.Runnable` interface



[1]



[2]



1st Method: Extending Thread Class

- Create a class by extending Thread class and override **run() method**:

```
class MyThread extends Thread
{
    public void run()
    {
        // thread body of execution
    }
}
```

- Create a thread:

```
MyThread thr1 = new MyThread();
```

- Start Execution of threads:

```
thr1.start();
```



Example

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println(" this thread is  
            running ... ");  
    }  
}  
  
class ThreadEx1 {  
    public static void main(String [] args ) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```



2nd method: Threads by implementing Runnable interface

- Create a class that implements the interface Runnable and override run() method:

```
class MyThread implements Runnable
{
    .....
    public void run()
    {
        // thread body of execution
    }
}
```

- Creating Object:

```
MyThread myObject = new MyThread();
```

- Creating Thread Object:

```
Thread thr1 = new Thread( myObject );
```

- Start Execution:

```
thr1.start();
```



Example

```
class MyThread implements Runnable {
    public void run() {
        System.out.println(" this thread is
                           running ... ");
    }
}

class ThreadEx2 {
    public static void main(String [] args ) {
        Thread t = new Thread(new MyThread());
        t.start();
    }
}
```

Overview of Thread Methods

- Thread-related methods
 - See API for more details (especially exceptions)
 - **run**
 - Does “work” of a thread – What does this mean?
 - Can be overridden in subclass of **Thread** or in **Runnable** object
 - **start**
 - Launches thread, then returns to caller
 - Calls **run**
 - Error to call **start** twice for same thread

Thread States: Life Cycle of a Thread

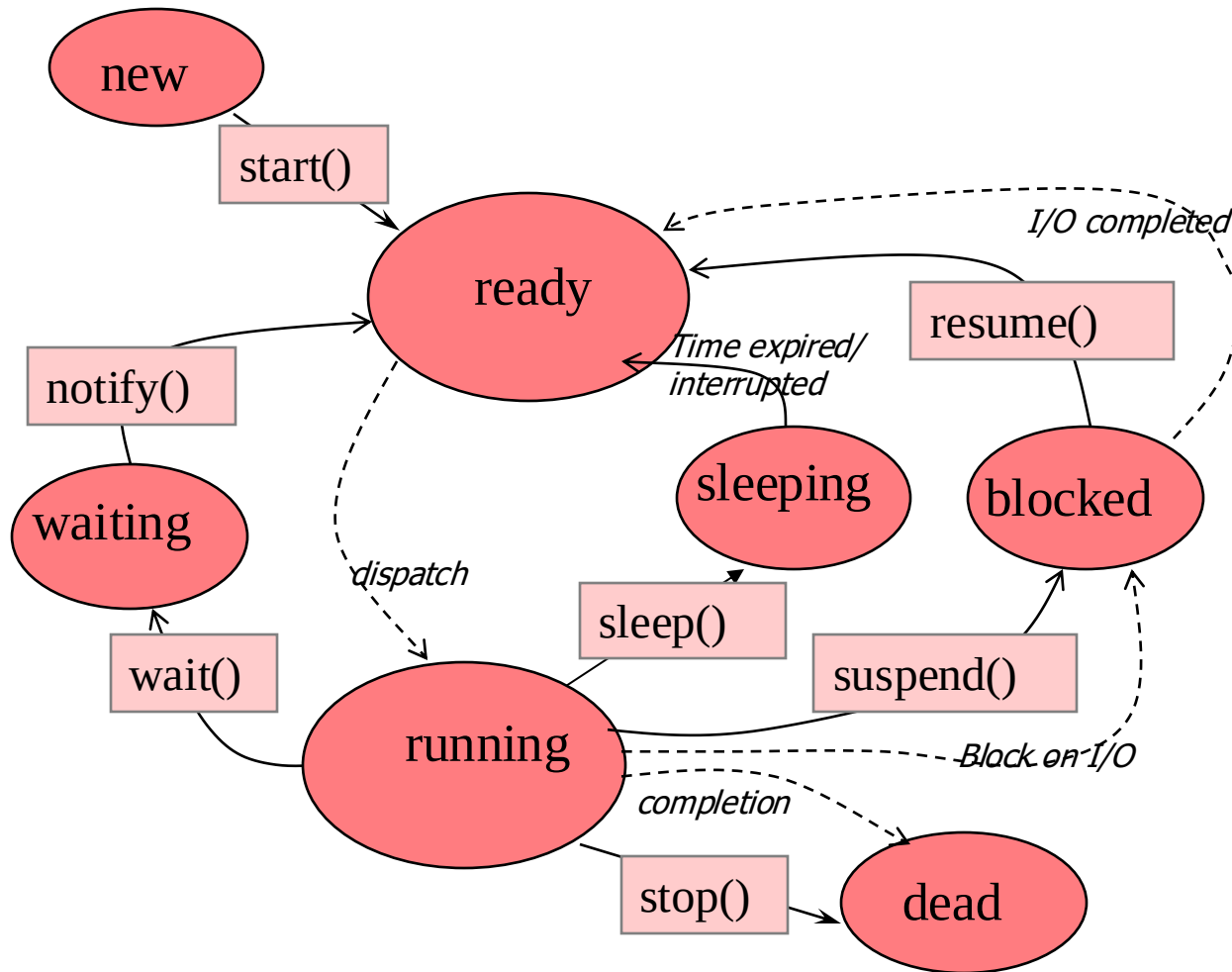
- Born state
 - Thread just created
 - When **start** called, enters ready state
- Ready state (runnable state)
 - Highest-priority ready thread enters running state
- Running state
 - System assigns processor to thread (thread begins executing)
 - When **run** completes or terminates, enters dead state
- Dead state
 - Thread marked to be removed by system
 - Entered when **run** terminates or throws uncaught exception

Other thread states

- Blocked state
 - Entered from running state
 - Blocked thread cannot use processor, even if available
 - Common reason for blocked state - waiting on I/O request
- Sleeping state
 - Entered when **sleep** method called
 - Cannot use processor
 - Enters ready state after sleep time expires
- Waiting state
 - Entered when **wait** called in an object thread
 - One waiting thread becomes ready when object calls **notify**
 - **notifyAll** - all waiting threads become ready



Life Cycle of Thread



A Program with Three Java Threads

- Implement a program that creates 3 threads

Three threads example

```
class A extends Thread
{
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println("\t From ThreadA: i= "+i);
        }
        System.out.println("Exit from A");
    }
}

class B extends Thread
{
    public void run()
    {
        for(int j=1;j<=5;j++)
        {
            System.out.println("\t From ThreadB: j= "+j);
        }
        System.out.println("Exit from B");
    }
}
```

Three threads example

```
class C extends Thread
{
    public void run()
    {
        for(int k=1;k<=5;k++)
        {
            System.out.println("\t From ThreadC: k= "+k);
        }

        System.out.println("Exit from C");
    }
}

class ThreadTest
{
    public static void main(String args[])
    {
        new A().start();
        new B().start();
        new C().start();
    }
}
```

Consideration

- The three threads are not exactly interleaved.
- The thread scheduler gives no guarantee about the order in which threads are executed.
- Each thread runs for a small amount of time called **Time Slice**
- The order in which each thread gains control is somehow **random**

Terminating Threads

- In Java 1.1, the Thread class had a stop() method
 - One thread could terminate another by invoking its stop() method.
 - However, using stop() could lead to deadlocks
 - **The stop() method is now deprecated.** DO NOT use the stop method to terminate a thread
- The correct way to stop a thread is to have the run method terminated
 - Add a boolean variable which indicates whether the thread should continue or not
 - Provide a set method for that variable which can be invoked by another thread

Example



```
public class ThreadToStop extends Thread{
```

```
    private boolean exit = false;
```

```
    public void run(){
```

```
        while(!exit){
```

```
            System.out.println("Thread is running...");
```

```
        }
```

```
        System.out.println("Thread is stopped!");
```

```
    }
```

```
    public void stopThread(){
```

```
        exit = true;
```

```
    }
```

```
}
```



Set the
variable from
outside the
program

```
public static void main(String[] args) throws InterruptedException {
```

```
    ThreadToStop thread = new ThreadToStop();
```

```
    thread.start();
```

```
    Thread.sleep(1000);
```

```
    thread.stopThread();
```

```
}
```

Terminating threads – Interrupting method

- To notify a thread that it should clean up and terminate you can use the `interrupt` method

```
t.interrupt();
```

- This method set a Boolean variable in the thread data structure
- In your run method you can check for interruptions for each iteration

```
public void run() {  
    for (int i = 0; i<=Repetition && !Thread.interrupted(); i++)  
    {  
        // Do work  
    }  
    // Clean up  
}
```

Thread Priorities

Each thread is assigned a default priority of `Thread.NORM_PRIORITY` (default priority). You can reset the priority using `setPriority(int priority)`.

Some constants for priorities include

`Thread.MIN_PRIORITY`

`Thread.MAX_PRIORITY`

`Thread.NORM_PRIORITY`



Yield() and sleep()

- Sometimes a thread can determine that it has nothing to do
 - Sometimes the system can determine this. ie. waiting for I/O
- When a thread has nothing to do, it should not use CPU
 - This is called a busy-wait.
 - Threads in busy-wait are busy using up the CPU doing nothing.
 - Often, threads in busy-wait are continually checking a flag to see if there is anything to do.
- It is worthwhile to run a CPU monitor program on your desktop
 - You can see that a thread is in busy-wait when the CPU monitor goes up (usually to 100%), but the application doesn't seem to be doing anything.
- Threads in busy-wait should be moved from the Run queue to the Wait queue so that they do not hog the CPU
 - Use yield() or sleep(time)
 - Yield simply tells the scheduler to schedule another thread
 - Sleep guarantees that this thread will remain in the wait queue for the specified number of milliseconds.

Potential risk when a thread is sleeping

- A thread might sleep for so long that is no longer useful and should be terminated.
- We just saw that to terminate the thread we interrupt it.
- When a sleeping thread is interrupted an `InterruptedException` is generated
- **You need to catch this exception!**

```
public void run() {  
    try{  
        //Task Statements  
        //...  
        Thread.sleep(5000)  
    }  
    catch (InterruptedException exception)  
    {  
        e.printStackTrace();  
    }  
    Clean up if necessary  
}
```



Conclusion

We saw:

- what is a Thread
- how to create and start a Thread
- how to stop a Thread
- Different thread states

In the next lecture, we will learn how to synchronise threads